LAB ACTIVITY · SIMHEURISTICS · STOCHASTIC TEAM ORIENTEERING

A simheuristic for the team orienteering problem with stochastic travel times

The biased-randomised simheuristic of Juan et al. (2019), rebuilt in Python + Numba and run on the 141 feasible instances of Chao's p1–p3 at three levels of uncertainty

Jesús Inurria Porrero · Javier Salvatierra Corchado

Team orienteering with random travel times

m vehicles leave the origin and must reach the end depot; each customer pays its reward once.

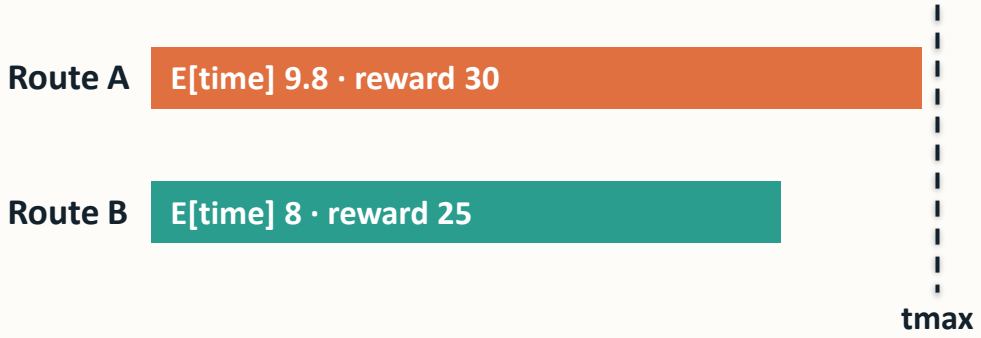
Each route has a time limit tmax, but the travel time of every arc is random.

A route that arrives after tmax loses all its reward.

Maximise the expected reward.

Already NP-hard when deterministic; now the value of a solution is an expectation, so it has to be estimated by simulation.

A MINIMAL EXAMPLE · ONE VEHICLE, TMAX = 10



DETERMINISTIC

A wins: 30 > 25

HIGH UNCERTAINTY

A late ~45 % of days: **≈ 16.5**
B almost never late: **≈ 25**

Under uncertainty, slack pays.

Illustrative example: the 45 % failure rate is assumed, not simulated. Fail rule from Juan et al. (2019), §3.

Chao's instances with lognormal travel times

TRAVEL TIME OF ARC (I, J) · JUAN ET AL. (2019), P. 1913

$$T_{ij} \sim \text{LogNormal}, \quad E[T_{ij}] = t_{ij}, \quad \text{Var}[T_{ij}] = c \cdot t_{ij}$$

t_{ij} = Euclidean distance; $T_{ij} = T_{ji}$; arcs independent between arcs and replicas (our assumption: the paper does not say).

THREE UNCERTAINTY LEVELS

Level	c	CV of one arc with t = 10
L (low)	0.05	7.1 %
M (medium)	0.25	15.8 %
H (high)	0.75	27.4 %

141

instances of Chao et al. (1996): p1, p2, p3, all below 35 nodes; tmax 3.8–55

× 3 levels

423 stochastic cases; each run with α = 0.25, 0.5, 0.75: 1 269 runs

On the fly

scenarios are drawn with a fixed seed and never stored; data/ is unchanged

Six p1 instances with no reachable customer are left out (48 + 33 + 60 remain). $\sigma^2 = \ln(1 + c/t)$, $\mu = \ln t - \sigma^2/2$. Reference: deterministic best-known solutions (BKS) of Dang et al. (2013).

Same c, very different uncertainty

Variances add along a route: $\mathrm{Var}[T_d] = c \cdot E_d$, so the relative sd is **$\sqrt{(c / E_d)}$**: the same c weighs more on short routes.

25.6 %

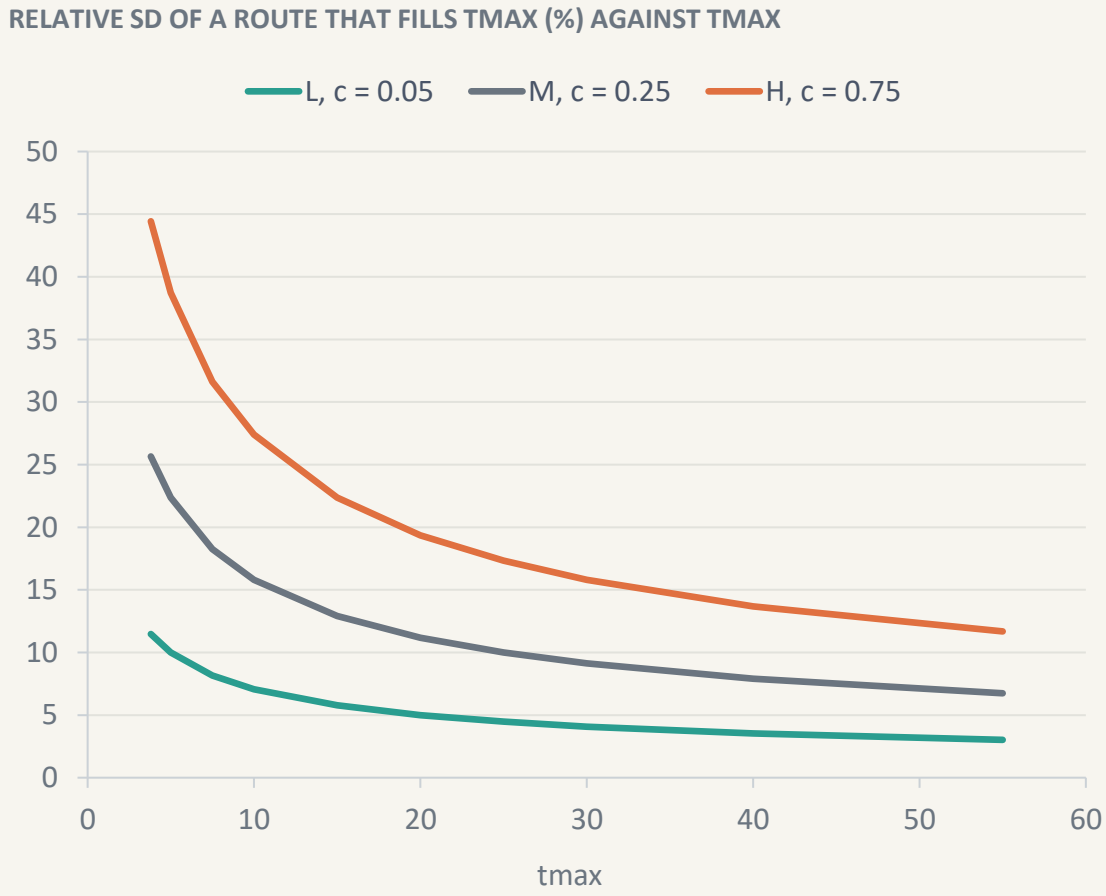
full route, c = 0.25, tmax = 3.8 (smallest in p1–p3)

6.7 %

the same c, tmax = 55 (largest in p1–p3)

47.18 %

of 20 000 days late for a route whose expected time equals tmax (4 arcs of 2.5, tmax = 10, c = 0.25): the lognormal's median lies below its mean



Slack is worth reward.

Chart: √(c / tmax) for a route that fills tmax, computed from the model (not a result). Independent arcs are our assumption.

The simheuristic: one run per (instance, c, α)

DETERMINISTIC PHASE · EXPECTED TIMES

51 candidates

1 greedy PJS + 50 PJS with geometric BR (β = 0.3)

→

Local search

2-opt + insertion of the highest-reward customer

→

Drop duplicates

same routes, in any order

→

Keep 7

highest deterministic reward (all if fewer than 7)

STOCHASTIC PHASE · MONTE CARLO

Fast simulation

200 days, common to the 7 (common random numbers)

→

Keep 3

highest fast mean

→

Intensive simulation

1 000 new days, common to the 3

→

Winner

highest intensive mean; reported: expected reward, % of failed routes

Biased randomisation explores; simulation evaluates. Two different random streams.

Per run, SeedSequence([2019, instance, c, α]) gives three independent generators: BR, fast and intensive simulation. PJS, local search and simulation loops are Numba kernels; random numbers come from numpy.

How the candidates are built

PREFERENCE OF ARC (I, J), BOTH DIRECTIONS (ORIGIN ≠ END DEPOT)

$$s'_{ij} = \alpha \cdot s_{ij} + (1 - \alpha) \cdot (u_i + u_j)$$

s_{ij} = time saved by joining the two routes

$u_i + u_j$ = reward of the two customers

PJS savings

One route origin → i → end per customer. Merge routes in order of s' while the expected time fits tmax. Keep the m routes with the most reward.

Geometric BR

Take the k-th remaining arc, $P(k) \approx \beta(1-\beta)^k$ with $\beta = 0.3$: the top arc 30 % of the time. Each construction gives a different, good solution.

Local search

2-opt with first improvement on every route, then insert the unvisited customer of highest reward that fits, at its cheapest position. Repeat until none fits.

Everything uses expected times, so merges and insertions fill tmax.

PJS of Panadero et al. (2021); BR as in Grasas et al. (2017). Check: the greedy PJS gives exactly the reward of our TOP assignment for the three α on the 141 instances.

Common days to compare, new days to measure

Fast: 200 days, the same for the 7

Goal: rank 7 candidates with few replicas.

With their own draws, one candidate could get lucky days and look better than it is.

With common days, a difference can only come from the routes.

Var(A − B) = Var(A) + Var(B) − 2 Cov(A, B)
A, B: 200-day mean rewards of two candidates

Intensive: 1 000 new days, shared by the 3

Goal: an unbiased estimate of the finalists.

They were chosen because they did well on the 200 days: re-using those days would inflate their mean (winner's curse).

1 000 days from another generator; they do not include the 200.

Each finalist is measured on days that did not select it.

Common is not the same as reused.

Common: candidates compared in the same phase see the same days — fair, and Cov > 0 lowers the variance.
Reused: the days that chose a solution also measure it — the estimate is inflated.

A small bias remains: the reported mean is the largest of three. With 1 000 replicas and similar finalists it is negligible; removing it would need a third simulation of the winner.

Two parameters from the paper, the rest ours

Parameter	Value	Origin
Uncertainty level c	0.05 / 0.25 / 0.75	paper, p. 1913
Replicas, fast / intensive	200 / 1 000	paper, p. 1912
α of the preference	0.25, 0.5, 0.75, all three run	ours: s_{ij} and $u_i + u_j$ have different scales
Biased randomisation	geometric, β = 0.3	ours, as in our TOP assignment
Stopping rule	50 BR constructions + 1 greedy	ours, for reproducibility (paper: time limit)
Filters	7 by deterministic reward → 3	ours, to keep a run within seconds
Local search	2-opt + insertion (TOP)	ours (paper: destruction–reconstruction + 2-opt)
Random numbers	common in fast, new in intensive	ours
Arcs	independent	ours: the paper does not state it
Seeds	one run per (instance, c, α)	ours: SeedSequence([2019, i, k, l])

The paper describes its simheuristic in one paragraph, without parameter values.

The greedy candidate guarantees that the best candidate in deterministic reward is no worse than PJS + local search, not that the final solution is: the winner is chosen by simulation.

α hardly matters; the uncertainty level does

GAP TO THE DETERMINISTIC BKS, MEAN / MEDIAN %, 141 INSTANCES PER CELL

α	c = 0.05 (L)	c = 0.25 (M)	c = 0.75 (H)
0.25	16.69 / 16.90	26.92 / 27.52	31.78 / 33.02
0.50	16.86 / 16.94	26.83 / 28.56	31.97 / 32.42
0.75	16.29 / 17.07	26.90 / 28.77	31.94 / 33.45

The mean gap of each fixed α varies by less than 0.6 points within each level. Keeping the best α per case lowers it by 1.3–2.2 points, partly through selection bias.

142 / 138 / 143

times α = 0.25 / 0.5 / 0.75 is the best of the 423 (instance, c) cases

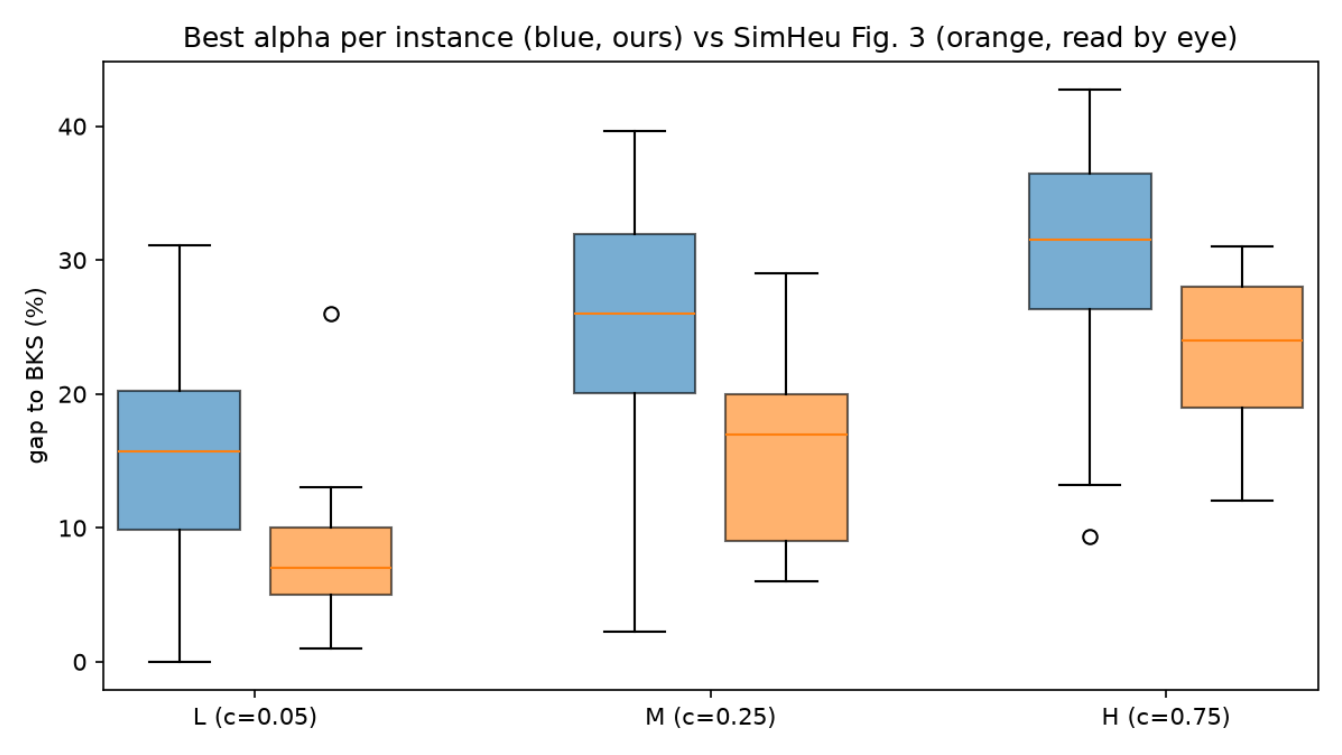
14.65 < 25.02 < 30.51

mean gap (%) with the best α per case: L < M < H

From here on: the best α of each (instance, c), 423 cases; levels are never mixed.

Gap = 100 (BKS − R) / BKS, with R the intensive mean of the winner and BKS the deterministic best-known solution. 1 269 runs over 141 instances.

Fig. 3: same ordering, 7–9 points higher



Blue: ours, best α. Orange: SimHeu-L/M/H of Fig. 3.

MEDIAN GAP % (BOX)

Level	Ours (Q1–Q3)	Paper ≈
L	15.69 (9.90–20.23)	7 (5–10)
M	26.02 (20.04–31.97)	17 (9–20)
H	31.56 (26.35–36.44)	24 (19–28)

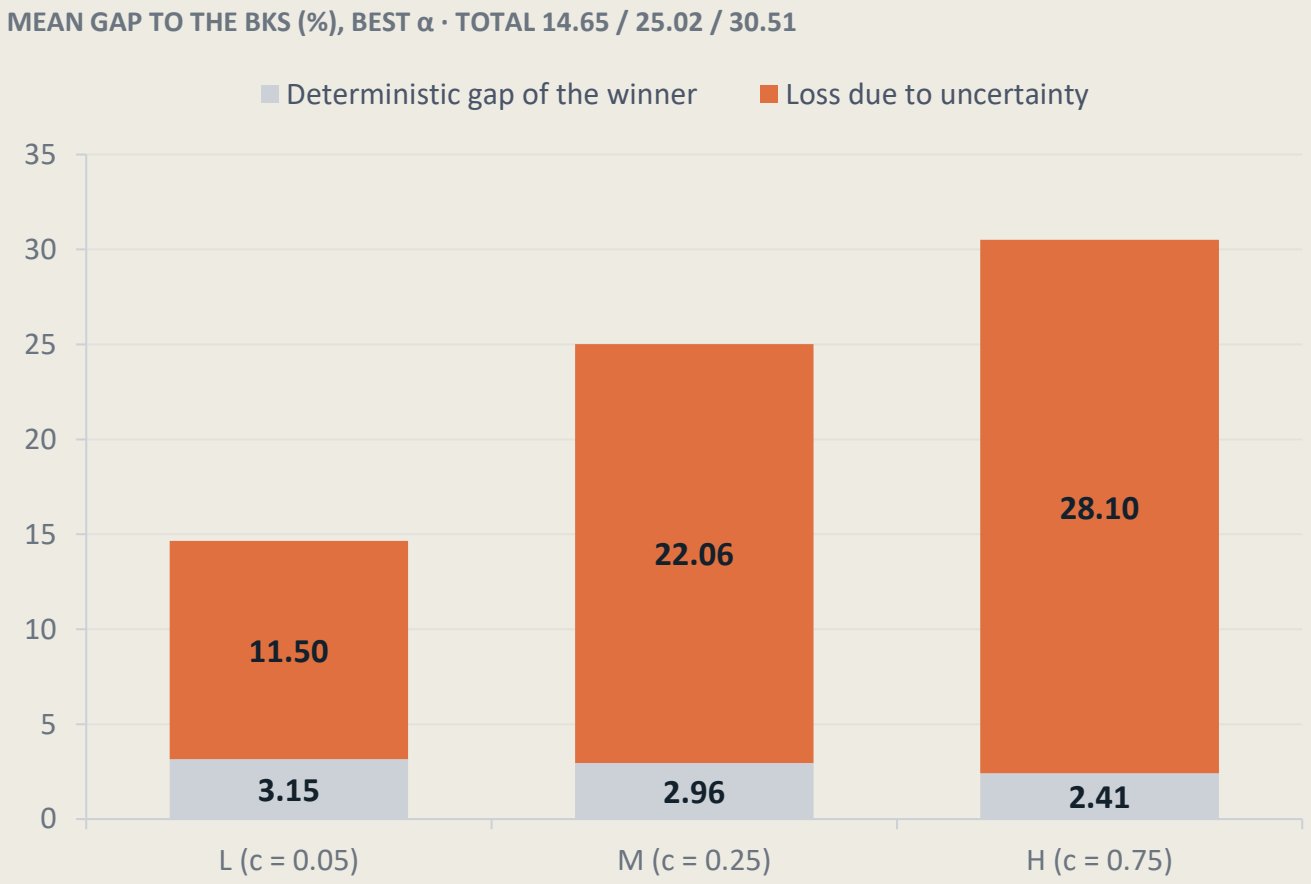
At L and M our lower quartile is about their upper one. Our deterministic gap is only 2.4–3.2 points.

PLAUSIBLE CAUSE (NOT CHECKED)

Juan et al. simulate inside the search loop and stop by time (up to ~400 s); we simulate only 7 candidates chosen with expected times.

Paper values read by eye from Fig. 3 (±1–2 points). The paper says neither which instances enter each box (we assume p1–p3, the only sets solved by the SAA) nor how the gap is computed.

Most of the gap is the price of uncertainty



≈ 80 % · 90 %

of the gap is loss due to uncertainty (L · M and H)

72 / 72 / 82

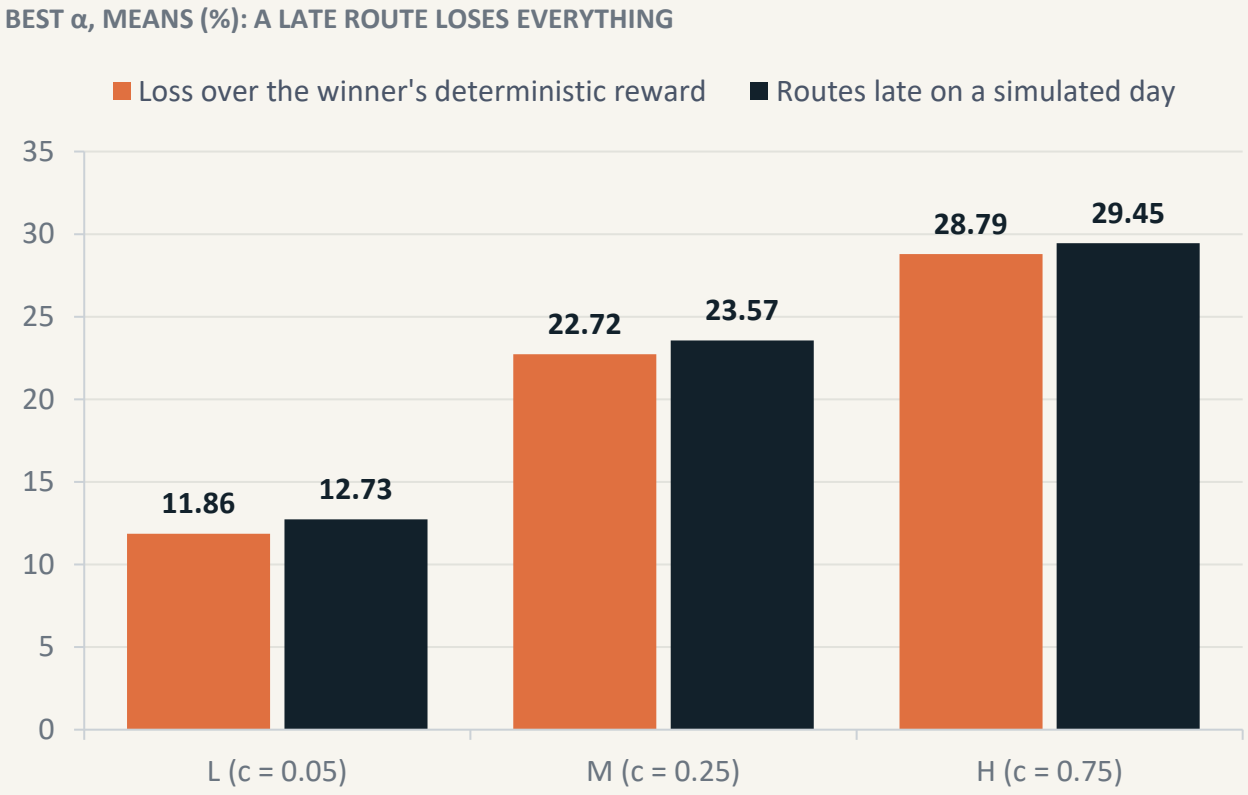
of 141 winners reach the BKS in deterministic terms: the median deterministic gap is 0

11.86 / 22.72 / 28.79 %

the same loss over the winner's own deterministic reward, 100 (det − R) / det

gap = deterministic gap 100 (BKS − det) / BKS + loss due to uncertainty 100 (det − R) / BKS, both over the BKS (means of the best-α rows). Part of the loss no solution can avoid.

Why: candidates are built up to tmax



0.93 · tmax

median expected time of the winners' routes, at every level

4 / 6 / 8 %

of those routes within 1 % of tmax, where a route fails almost half of the time

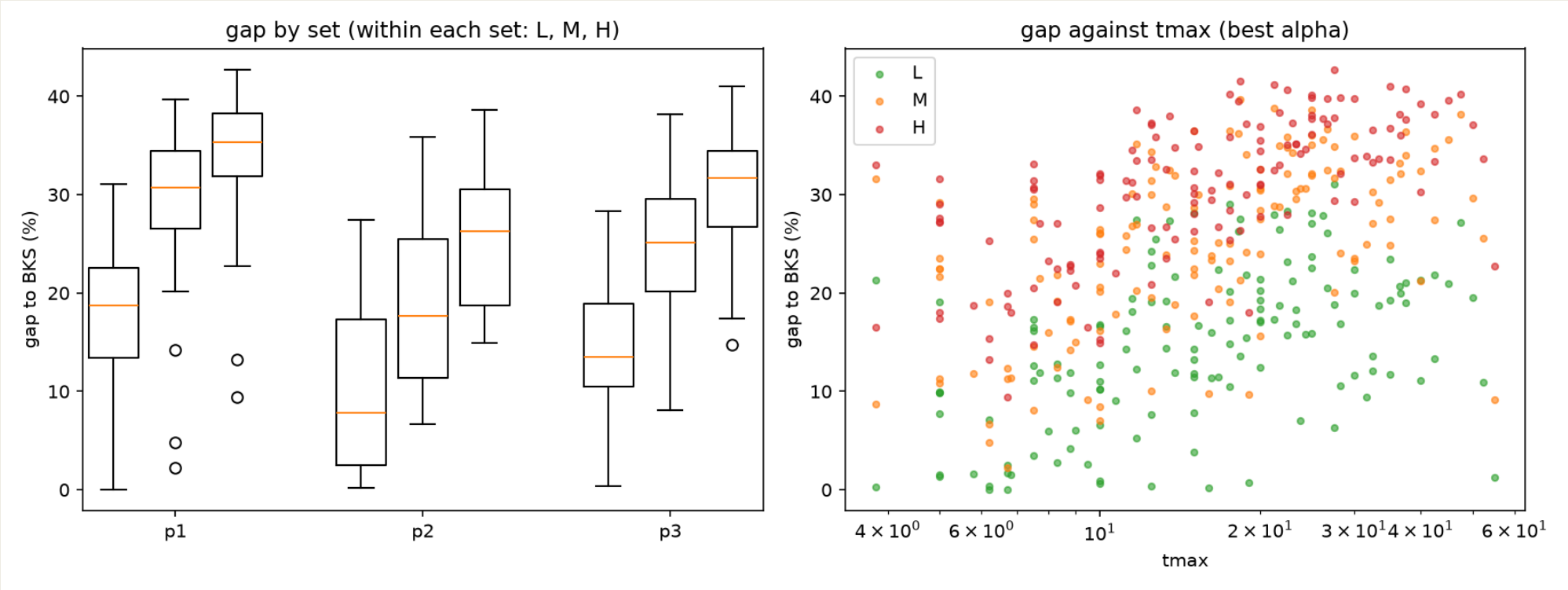
27.1 %

of the 1 269 runs have fewer than 7 distinct candidates, so the filter does not act (9.0 %: only one)

Merges and insertions fill tmax, and the filter keeps the fullest: simulation can only choose among routes built without slack.

Faithful to the paper, which also plans routes with expected times (its Eq. 3).

Larger tmax, larger gap — despite Var = c·t



0.49 / 0.56 / 0.67

Spearman of tmax and gap, L / M / H

10.4 / 19.3 / 25.2 %

mean gap in p2, the set with the smallest tmax (median 10)

0.56 / 0.54 / 0.59

Spearman of tmax and the deterministic gap, more than the loss (0.30 / 0.39 / 0.44)

Best α; loss due to uncertainty over the BKS. Set and tmax are confounded (p2 has only 21 nodes): an observation, not an explanation.

Fast enough: 1 269 runs in about 25 seconds

≈ 25 s

for all 1 269 runs on 12 threads of a 6-core AMD Ryzen 5 5500U

0.05–0.23 s

per run, after loading the Numba kernels

0.07 · 0.16 s

mean per run in p2 · in p1 and p3 (0.16–0.17)

Python 3.14 + NumPy + Numba

PJS merges, local search and the loop over replicas are Numba kernels; random numbers come from NumPy. Each day is an N × N matrix of symmetric times read by all the candidates of a phase. 18 sanity checks.

The paper's Fig. 4

A few seconds to about 400 s, in Java on a 3.7 GHz Xeon with a time limit. Different language, machine and stopping rule: only a qualitative comparison.

Cheap enough to run every α at every level instead of tuning one.

Time of one call to the simheuristic once the Numba kernels are loaded; a process pool spreads the runs over the 12 threads.

Limitations and next step: a safety margin

Limitations

No SAA: it needs a commercial solver (CPLEX), so only the simheuristic is compared.

Fig. 3 read by eye: we assume its boxes cover p1–p3, and the paper does not give the gap formula.

Two small selection biases: the reported mean is the largest of three finalists, and the best of three α is kept with that mean.

Candidates built with expected times, as in the paper: no slack.

Next step: a safety margin

Build with tmax − δ while simulating with the true tmax. δ from:

1. **A grid:** e.g. $tmax - \delta = k \cdot tmax$, $k \in \{0.80, 0.85, \ldots, 1\}$; simulation picks the best.
2. **The route variance:** accept a route only if $E_d + z \sqrt{(c\, E_d)} \leq tmax$: a margin per route and per c.
3. **Adaptive:** tuned to the failure rate of the fast simulation.

Motivated by 13–29 % of winning routes arriving late, more as c grows.

TAKEAWAY

Simulation can only choose among routes built without slack.

WHAT WE FOUND

Mean gaps 14.65 / 25.02 / 30.51 % (L / M / H): the ordering of Juan et al., medians 7–9 points above theirs (read by eye).

Winners usually match the BKS deterministically, but the median route fills 0.93 tmax and 13–29 % of routes arrive late.

A fraction of a second per run; α hardly matters.

WORTH ASKING ABOUT

No SAA: only the simheuristic is compared.

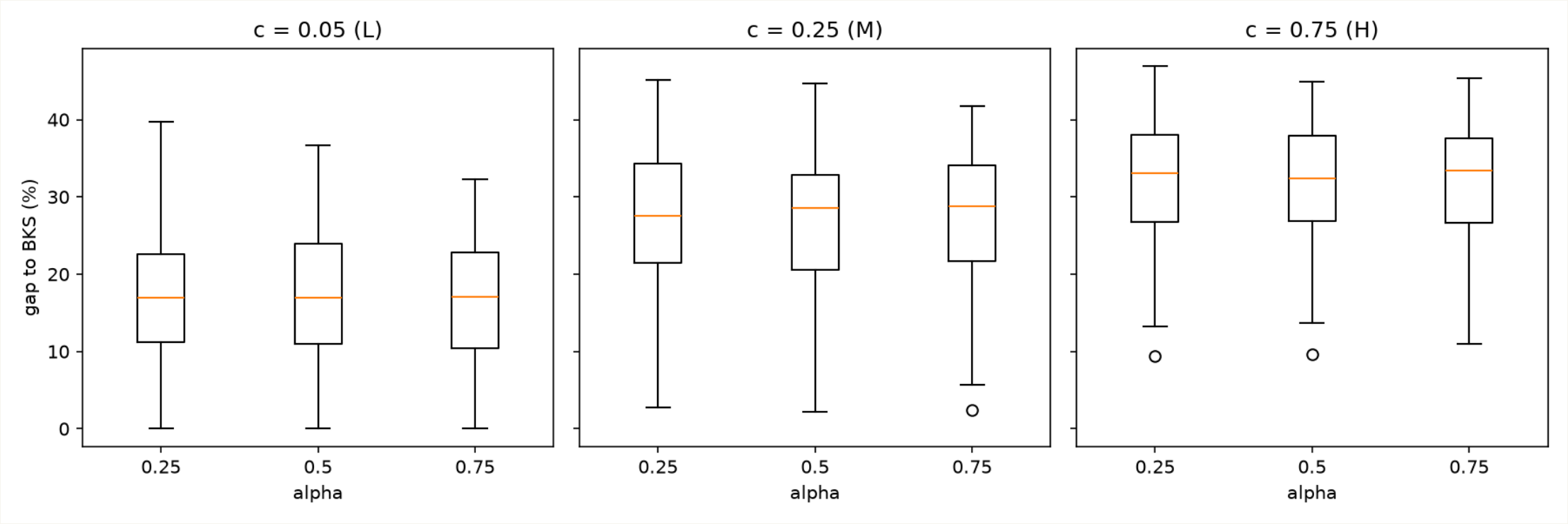
Fig. 3 read by eye; instances per box assumed.

Next: a safety margin, tmax − δ.

Thank you — questions?

BACKUP

Gap by α and level, all 1 269 runs



results/gap_alpha_c_matrix.csv; 141 instances per box. Medians 16.9–17.1 (L), 27.5–28.8 (M), 32.4–33.5 (H).